Table of Contents

1. Introduction
    1.1. Scope and Assumptions
2. How To Read This Document
3. Document Health Snapshot
4. Terms and Definitions
    4.1. Diagram Color and Shape Legend
5. Architecture Intent View
    5.1. What This View Answers
    5.2. System Boundary Diagram
    5.3. System Decomposition Diagram
    5.4. Functional Group Manhattan Matrix
    5.5. Functional Unit Dependency Diagram
    5.6. Functional Groups and Units (View Scoped)
6. Communication View
    6.1. What This View Answers
    6.2. Coverage Gaps
    6.3. Recommended Next Evidence Additions
    6.4. Communication Path Graph
    6.5. Functional Groups and Units (View Scoped)
7. Deployment View
    7.1. What This View Answers
    7.2. Coverage Gaps
    7.3. Recommended Next Evidence Additions
    7.4. Deployment Artifact Relationship Graph
    7.5. Platform Operations Graph
    7.6. Functional Groups and Units (View Scoped)
8. Security View
    8.1. What This View Answers
    8.2. Coverage Gaps
    8.3. Recommended Next Evidence Additions
    8.4. Attack Path Diagram
    8.5. Security Logging and Observability
    8.6. Functional Groups and Units (View Scoped)
9. Traceability View
    9.1. What This View Answers
    9.2. Coverage Gaps
    9.3. Recommended Next Evidence Additions
    9.4. Traceability Evidence Graph
    9.5. Requirement-to-Unit Mapping
    9.6. Requirement-to-Unit-to-Evidence Coverage
10. Traceability Appendix
    10.1. Requirement Details
    10.2. Verification Inventory

10.3. Verification Result Mapping
11. Reference Index
11.1. Authored References
11.2. Catalog References
11.3. Inferred Runtime References
11.4. Inferred Code References
11.5. Verification References

---

1. Introduction

This document describes the `payments` architecture using authored functional design and inferred runtime/code realization. It is intended to help product, `platform`, security, and implementation engineers reason about one shared model. Functional design is kept stable while realization details are inferred from infrastructure and source artifacts.

1.1. Scope and Assumptions

- `Functional Groups` and `Functional Units` are the stable authored design backbone.
- `Runtime elements` and `code elements` are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- `Requirements` and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
3. Traceability View: requirement mapping, evidence coverage, and confidence.
4. Reference Appendix: supporting reference IDs and inferred-source links.

3. Document Health Snapshot

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Architecture Intent View (`VIEW-ARCHITECTURE-INTENT`)	stable	Stable authored capability boundaries and ownership split; changes are intentional and infrequent.	6	6	0	high	Defaults authored units to covered; use authoredStatus as the primary publication signal.
Communication View (`VIEW-COMMUNICATION`)	in-review	Communication semantics are partially inferred and still being refined with explicit protocol and interaction metadata.	4	3	1	medium	Counts a unit as covered when direct inferred runtime evidence exists.
Deployment View (`VIEW-DEPLOYMENT`)	in-review	Deployment chain and control-plane mapping are mostly present, but ownership evidence remains incomplete for some units.	6	5	1	high	Counts a unit as covered when direct inferred runtime evidence exists.
Security View (`VIEW-SECURITY`)	draft	Threat mapping is present for key vectors, but control expectations and coverage breadth are still evolving.	6	6	2	high	Counts a unit as covered when explicit authored attack-vector mapping exists.
Traceability View (`VIEW-TRACEABILITY`)	in-review	Requirement-to-unit linkage is present; cross-layer ownership consistency still needs tightening.	6	5	4	high	Counts a unit as covered when direct inferred runtime or code evidence exists.

4. Terms and Definitions

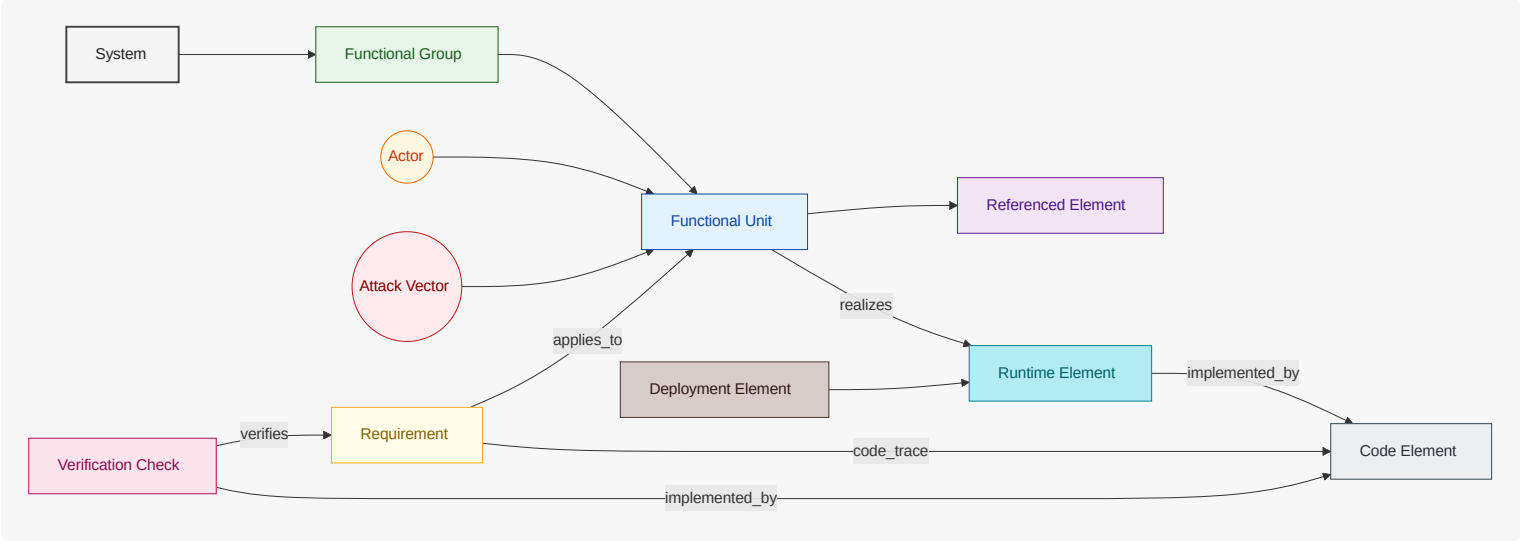
Term	Definition
`3DS verification is required` (COND-3DS-REQUIRED)	Condition indicating strong customer authentication must be performed. Aliases:3DS required
`actor` (EM-ACTOR)	A person or operational role that interacts with functional units.
`attack vector` (EM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
`authored mapping` (EM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.
`bank gateway endpoint` (REF-BANK-GATEWAY-ENDPOINT)	External banking endpoint used for card authorization and decline decisions. Aliases:bank API
`bank link is available` (STATE-BANK-LINK-UP)	Connectivity state showing bank gateway communication is currently healthy. Aliases:bank link up
`bank link is unavailable` (EVT-BANK-LINK-UNAVAILABLE)	Event raised when payment engine cannot reach the external bank gateway. Aliases:bank link down
`checkout handling` (FU-CHECKOUT)	Handles checkout session start and customer-facing payment messaging. Aliases:checkout
`checkout is active` (STATE-CHECKOUT-ACTIVE)	Processing state where the user is actively submitting or confirming payment. Aliases:checkout active
`cluster provisioning` (FU-CLUSTER-PROVISIONING)	Provides cluster-level platform setup and environment bootstrap. Aliases:day1 provisioning
`code element` (EM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
`compromised dependency` (AV-COMPROMISED-DEPENDENCY)	Supply-chain compromise in runtime image or code dependency. Aliases:dependency compromise
`customer` (ACT-CUSTOMER)	End user submitting payment details during checkout. Aliases:buyer
`decline reason` (DATA-DECLINE-REASON)	Structured reason code or message explaining why authorization failed. Aliases:decline code, decline reasons
`fraud check is enabled` (FEAT-FRAUD-CHECK)	Feature flag controlling whether risk scoring is required before authorization. Aliases:fraud check
`fraud detection system` (SYS-FRAUD-DETECTION-SYSTEM)	Fraud scoring and review subsystem participating in payment decisions. Aliases:the fraud detection system

Term	Definition
`fraud evaluation` (FG-FRAUD)	Functional group for risk scoring and fraud review capabilities. Aliases:fraud domain
`fraudulent transaction pattern` (AV-FRAUDULENT-TRANSACTION-PATTERN)	Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals. Aliases:fraud abuse, payment fraud
`functional group` (EM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
`functional unit` (EM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.
`gitops operations` (FU-GITOPS-OPERATIONS)	Operates declarative delivery and release management flows. Aliases:day2 operations
`helm platform` (REF-HELM-PLATFORM)	Runtime packaging and deployment substrate for workloads. Aliases:helm
`inferred mapping` (EM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
`malicious api request` (AV-MALICIOUS-API-REQUEST)	Crafted payload attempting to bypass validation or authorization logic. Aliases:malicious request
`normal mode is active` (MODE-NORMAL)	Standard operating mode with full external dependency usage enabled. Aliases:normal mode, standard operating mode
`payment authorization` (FU-PAYMENT-AUTHORIZATION)	Coordinates authorization flow and bank gateway interaction. Aliases:auth
`payment authorization is requested` (EVT-PAYMENT-AUTH-REQUESTED)	Event raised when checkout asks payment engine to authorize a transaction. Aliases:authorization requested, payment auth requested
`payment id` (DATA-PAYMENT-ID)	Stable identifier used to correlate a payment across services and records.
`payment is declined` (EVT-PAYMENT-DECLINED)	Event indicating authorization was rejected and customer decline handling must execute. Aliases:decline, payment declined
`payment review is pending` (STATE-REVIEW-PENDING)	Transaction state requiring manual support review before final disposition. Aliases:review pending, review state
`payments` (FG-PAYMENTS)	Functional group for checkout and authorization capabilities. Aliases:payments domain
`payments system` (SYS-PAYMENTS-SYSTEM)	Payment processing system-of-interest covered by this architecture. Aliases:the payments system

Term	Definition
`platform` (FG-PLATFORM)	Functional group for provisioning and platform operations capabilities. Aliases:platform engineering
`platform operator` (ACT-PLATFORM-OPERATOR)	Role responsible for GitOps and platform lifecycle operations. Aliases:platform ops
`postgresql client driver` (REF-POSTGRES-DRIVER)	Third-party library used for application data persistence connectivity. Aliases:postgres driver
`referenced element` (EM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
`replayed authorization callback` (AV-REPLAYED-AUTH-CALLBACK)	Replay of a valid callback to induce duplicate or unauthorized state transitions. Aliases:callback replay
`requirement` (EM-REQUIREMENT)	A structured requirement statement that defines expected system behavior and maps to functional ownership.
`risk score` (DATA-RISK-SCORE)	Numeric fraud score produced by risk evaluation logic.
`risk score is high` (EVT-RISK-HIGH)	Event emitted when computed fraud risk exceeds the configured high-risk threshold. Aliases:high risk, high-risk classification, high-risk classifications
`risk scoring` (FU-RISK-SCORING)	Evaluates transaction risk and returns score classification. Aliases:fraud scoring
`risk threshold is exceeded` (COND-RISK-THRESHOLD-EXCEEDED)	Condition indicating risk score crossed a policy threshold for escalation. Aliases:risk above threshold
`runtime element` (EM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
`support agent` (ACT-SUPPORT)	Operations role that investigates declines and high-risk review cases. Aliases:support
`support agent is notified` (EVT-SUPPORT-NOTIFIED)	Event indicating manual review notification was sent to support operations. Aliases:support notified
`support review` (FU-SUPPORT-REVIEW)	Supports manual review and escalation handling for risky transactions. Aliases:manual review
`upward linking` (EM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.
`verification check` (EM-VERIFICATION-CHECK)	An inferred or authored verification artifact that validates requirement behavior with test evidence.

4.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



5. Architecture Intent View

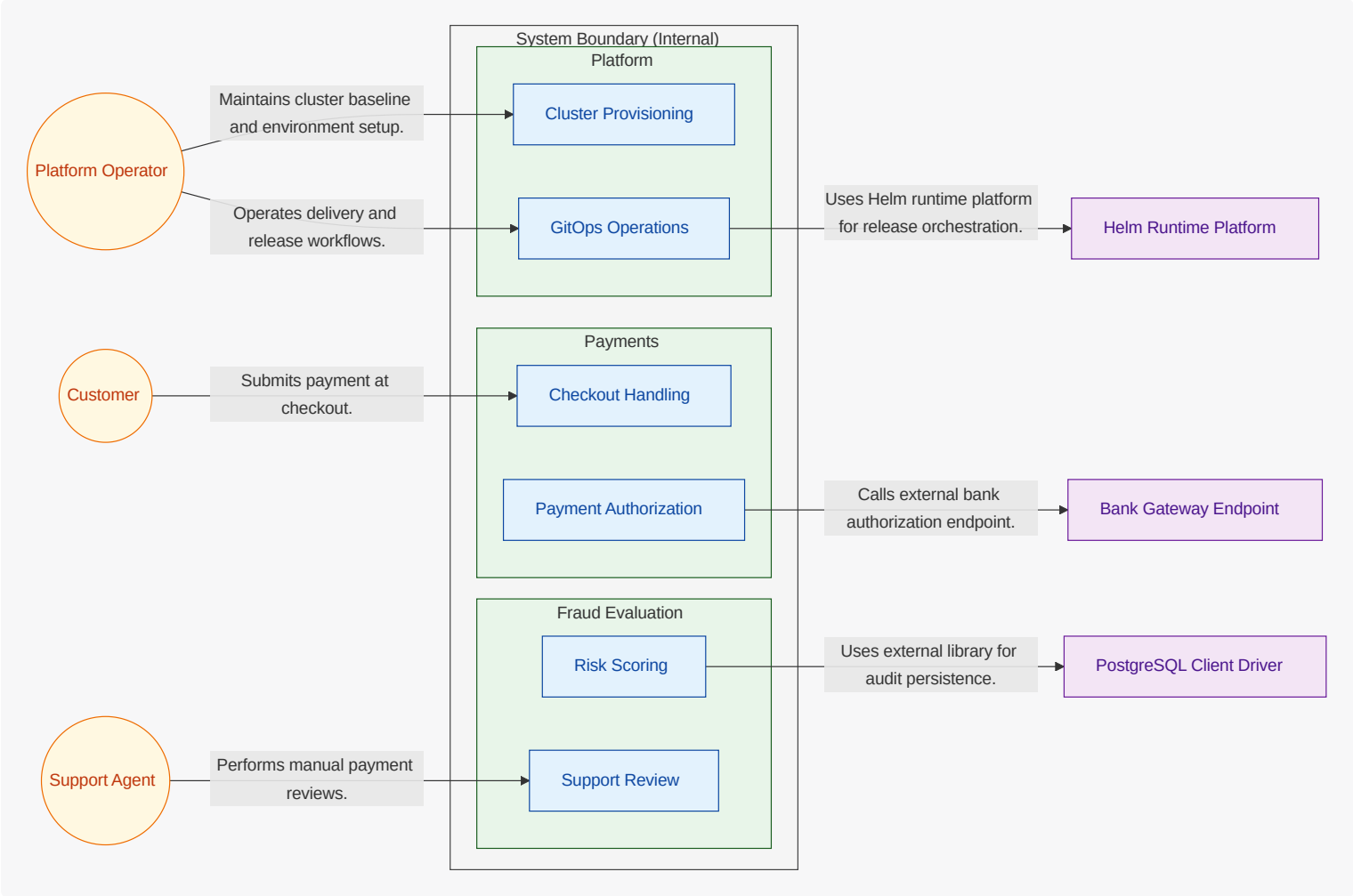
Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

5.1. What This View Answers

- What are the main capability areas?
- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

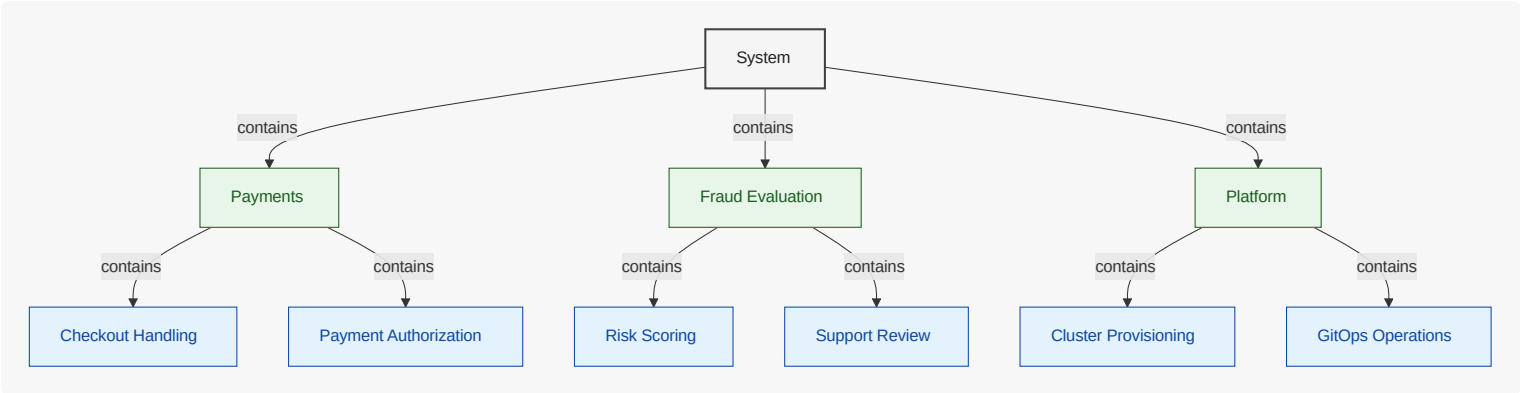
5.2. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the `System Boundary (Internal)` container, while actors and referenced dependencies are shown outside the boundary.



5.3. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.



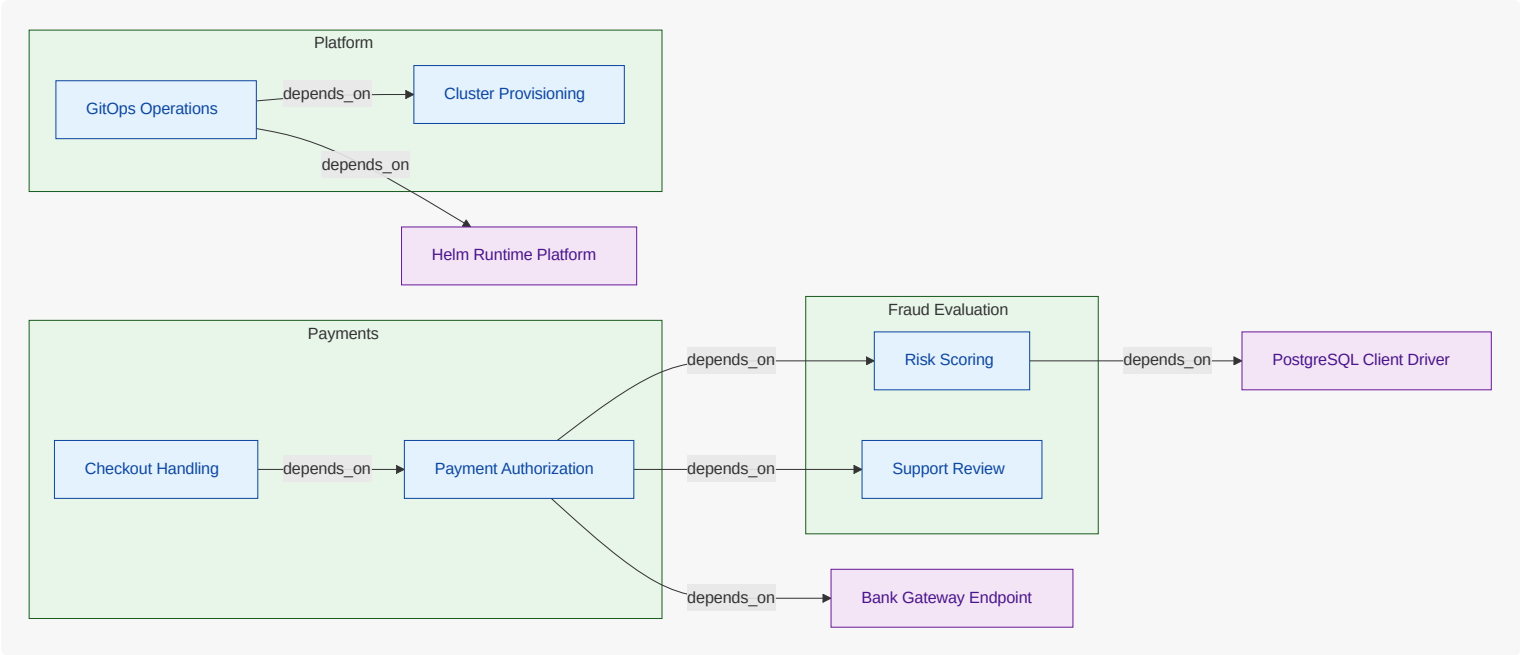
5.4. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

Checkout Handling	Risk Scoring	Cluster Provisioning
Payment Authorization	Support Review	GitOps Operations
Payments	Fraud Evaluation	Platform

5.5. Functional Unit Dependency Diagram

What this diagram shows: dependency paths between functional units (grouped in Functional Group containers) and key referenced elements. Arrow meaning: `A -→|depends_on| B` means unit `A` requires behavior or data from `B` to fulfill its responsibility.



5.6. Functional Groups and Units (View Scoped)

5.6.1. Payments (FG-PAYMENTS)

Core payment `checkout` and authorization domain.

The `payments` group defines the product-facing payment journey from user intent to final payment outcome. It owns the consistency of `customer` messaging and transaction decisions when a payment is attempted, declined, retried, or temporarily unavailable. The group keeps `customer` flow stable while depending on fraud and external bank capabilities.

Checkout Handling (FU-CHECKOUT)

`Checkout handling` is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear `customer` feedback for each outcome. It delegates decision logic to `payment authorization` while protecting the user experience boundary.

Owned Responsibility

functional responsibility for `checkout handling`; includes decision and orchestration flow to `Payment Authorization`

Key Collaborators

`Payment Authorization`

Linked Requirements

`REQ-PAY-001`, `REQ-PAY-003`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-008`

Payment Authorization (FU-PAYMENT-AUTHORIZATION)

`Payment authorization` orchestrates the final transaction decision path. It coordinates `fraud scoring`, external bank interactions, and escalation to `support review` when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable.

Owned Responsibility

functional responsibility for `payment authorization`; includes decision and orchestration flow to `Bank Gateway Endpoint` and decision and orchestration flow to `Risk Scoring`

Key Collaborators

`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`

Linked Requirements

`REQ-PAY-001`, `REQ-PAY-004`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-009`

5.6.2. Fraud Evaluation (FG-FRAUD)

`Risk scoring` and fraud audit domain.

The `fraud evaluation` group exists to reduce unsafe approvals while preserving throughput for legitimate `payments`. It combines automated `risk scoring` with manual `support review` so uncertain transactions move through a controlled escalation path. The group ensures `risk` decisions are explainable, auditable, and operationally actionable.

Risk Scoring (FU-RISK-SCORING)

`Risk scoring` computes and classifies transaction `risk` before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior.

Owned Responsibility

functional responsibility for `risk scoring`; includes decision and orchestration flow to `PostgreSQL Client Driver`

Key Collaborators

`PostgreSQL Client Driver`

Linked Requirements

`REQ-PAY-002`, `REQ-PAY-005`

Support Review (FU-SUPPORT-REVIEW)

`Support review` handles manual decisions for escalated or ambiguous payment cases. It gives `support` operators context to approve, reject, or request additional verification in a controlled flow. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable.

Owned Responsibility

functional responsibility for `support review`

Key Collaborators

none

Linked Requirements

`REQ-PAY-004`, `REQ-PAY-009`

5.6.3. Platform (FG-PLATFORM)

`Cluster provisioning` and `GitOps operations` domain.

The `platform` group provides the runtime and deployment foundation used by all product capabilities. It owns environment provisioning, namespace and cluster boundaries, and continuous reconciliation behavior. The group enables safe delivery and reliable operations without owning payment business logic.

Cluster Provisioning (FU-CLUSTER-PROVISIONING)

`Cluster provisioning` creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations.

Owned Responsibility

functional responsibility for `cluster provisioning`

Key Collaborators

none

Linked Requirements

no explicit authored `requirement` trigger

GitOps Operations (FU-GITOPS-OPERATIONS)

`GitOps operations` continuously reconciles intended release state with actual runtime state. It governs rollout flow, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability.

Owned Responsibility

functional responsibility for `gitops operations`; includes decision and orchestration flow to `Cluster Provisioning` and decision and orchestration flow to `Helm` Runtime `Platform`

Key Collaborators

`Cluster Provisioning`, `Helm` Runtime `Platform`

Linked Requirements

no explicit authored `requirement` trigger

6. Communication View

Show who communicates with whom across key request paths, including interface edges and dependency handoffs.

6.1. What This View Answers

- Who talks to whom and over which interfaces?
- Which edges are synchronous, asynchronous, event-driven, or callback-oriented?
- Which communication paths are critical to the main request flow?

6.2. Coverage Gaps

Coverage summary: 3/4 units have direct evidence coverage in this view.

- `Support Review` has no direct inferred runtime/deployment evidence yet.

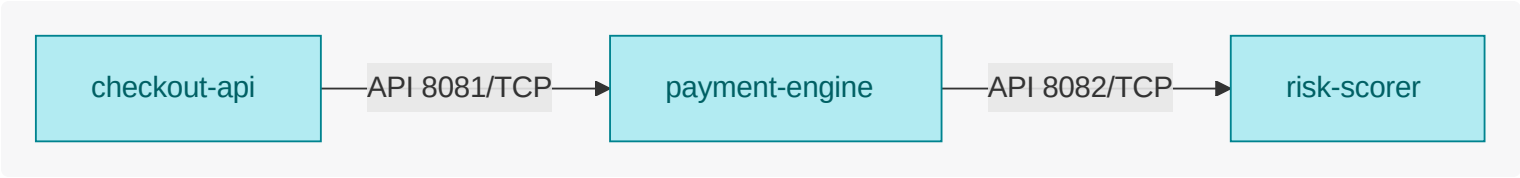
6.3. Recommended Next Evidence Additions

- Add explicit communication metadata (protocol, sync/async/event/callback) to `authored mappings` where currently implicit.
- Expose service protocol/port metadata in `Helm` or manifest values to enrich interface-edge inference.
- Capture missing communicating runtime units by adding discoverable workload artifacts under scanned inference roots.

6.4. Communication Path Graph

What this diagram shows: inferred unit-to-unit request/call flow, including available interface/port labels.

> **NOTE** This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or interface details indicate missing derived communication evidence, not necessarily missing behavior.



6.5. Functional Groups and Units (View Scoped)

6.5.1. Payments (FG-PAYMENTS)

`payments` runs the public request path and keeps transaction context intact across runtime hops. The runtime design emphasizes low latency, deterministic request sequencing, and clear operational signals when dependencies slow down or fail.

Checkout Handling (FU-CHECKOUT)

`checkout handling` is the ingress runtime boundary and forwards normalized requests downstream. Runtime design emphasizes stable request contracts, context propagation, and graceful behavior under burst traffic and partial downstream degradation.

Inbound Interfaces

Interface	Description
`Customer`	Submits payment at `checkout`.

Outbound Interfaces

Interface	Description
`Payment Authorization`	Delegates `payment authorization`.

Communication Evidence

runtime: `payments`, `payments/checkout-api` | code modules: `checkout_api`

Payment Authorization (FU-PAYMENT-AUTHORIZATION)

`payment authorization` coordinates critical runtime dependencies and enforces decision-order determinism. Runtime design prioritizes idempotent transitions, explicit branch visibility, and controlled error normalization.

Inbound Interfaces

Interface	Description
`Checkout Handling`	Delegates `payment authorization`.

Outbound Interfaces

Interface	Description
`Bank Gateway Endpoint`	Calls external bank authorization endpoint.
`Risk Scoring`	Requests `fraud scoring` before final authorization.
`Support Review`	Escalates high-risk `payments` for `manual review`.

Communication Evidence

runtime: `payments/payment-engine` | code modules: domain, `payment_engine`

6.5.2. Fraud Evaluation (FG-FRAUD)

`fraud evaluation` runs as a critical dependency on the authorization path. Runtime behavior must return stable contracts quickly while preserving enough evidence for operator diagnosis and audit when `high-risk classifications` occur.

Risk Scoring (FU-RISK-SCORING)

`risk scoring` runs as a synchronous dependency with strict response-shape stability. Runtime design favors deterministic outputs, bounded latency, and clear degradation semantics for callers.

Inbound Interfaces

Interface	Description
`Payment Authorization`	Requests `fraud scoring` before final authorization.

Outbound Interfaces

Interface	Description
`PostgreSQL Client Driver`	Uses external library for audit persistence.

Communication Evidence

runtime: `risk`, `risk/risk-scorer` | code modules: `risk_scorer`, `support`

Support Review (FU-SUPPORT-REVIEW)

`support review` runtime optimizes for operator clarity and decision traceability over throughput. Runtime design emphasizes state visibility, reversible transitions where allowed, and reliable action logging.

Inbound Interfaces

Interface	Description
`Payment Authorization`	Escalates high-risk `payments` for `manual review`.
`Support Agent`	Performs manual payment reviews.

Outbound Interfaces

none

Communication Evidence

authored unit with no direct derived runtime/code evidence yet

7. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This view emphasizes deployment relationships and `platform` operations.

7.1. What This View Answers

- How does authored intent become deployed runtime?
- What control-plane objects manage deployable units?
- Where are `platform` ownership boundaries and missing ownership signals?

7.2. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view.

- `Support Review` has no direct inferred runtime/deployment evidence yet.

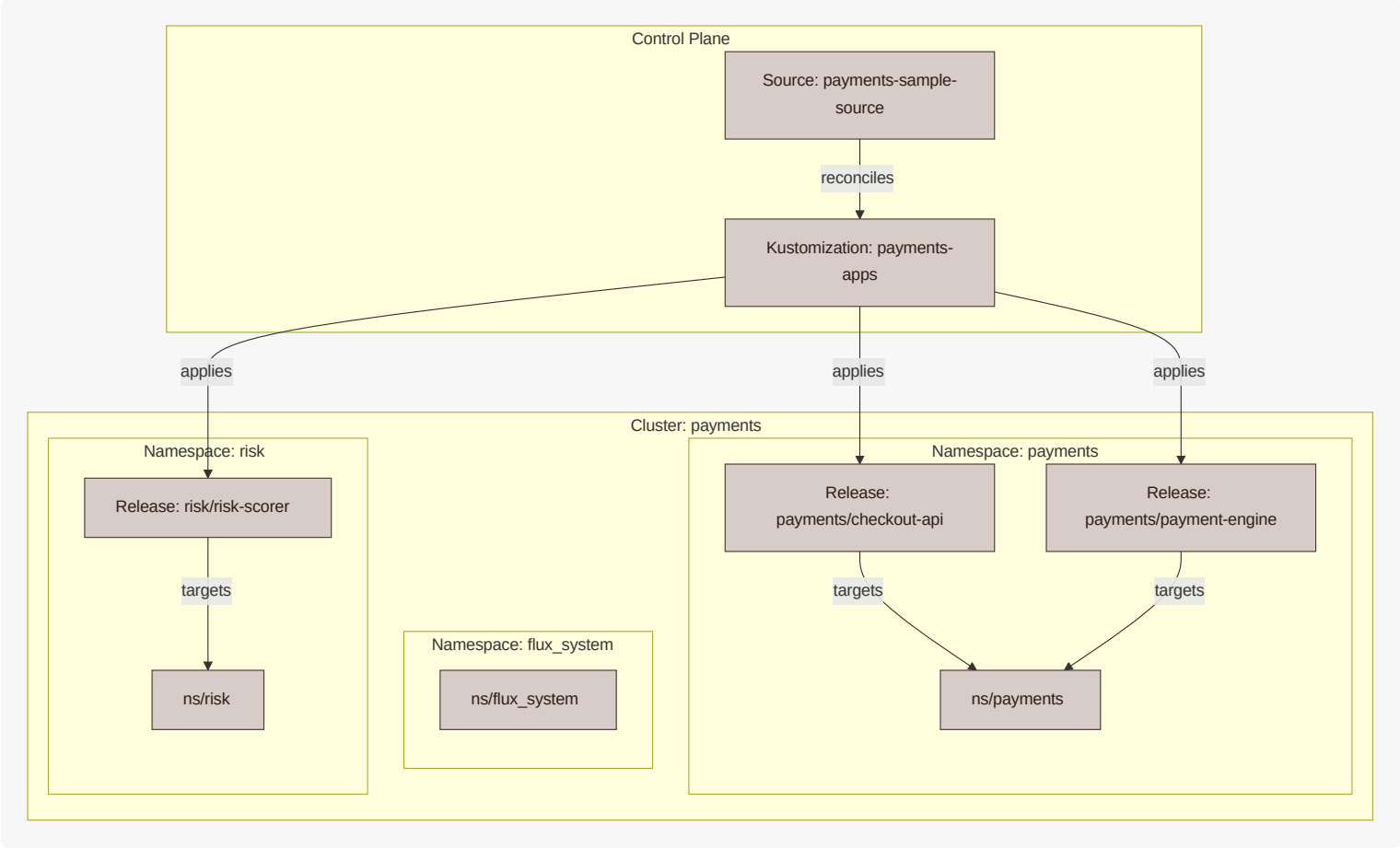
7.3. Recommended Next Evidence Additions

- Add owner metadata to Flux/Kustomization/`Helm` objects for clearer `platform` ownership mapping.
- Ensure deployment control artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking.

7.4. Deployment Artifact Relationship Graph

What this diagram shows: how source/control-plane artifacts connect to releases, namespaces, workloads, and target cluster context.

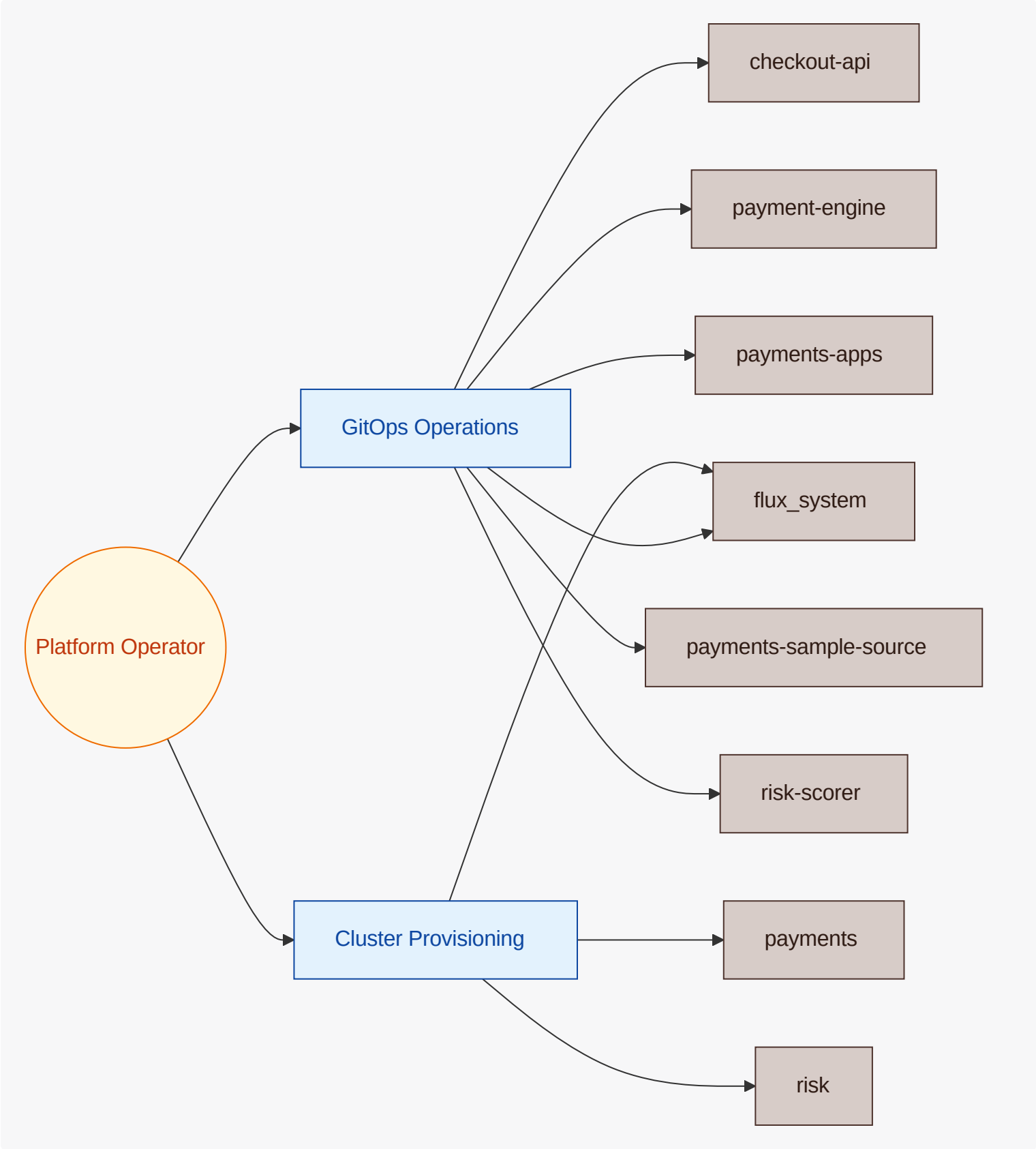
> **NOTE**
> This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete. Namespace and cluster boundaries are rendered as containers when target/placement relationships are inferred. Node labels include role hints (for example `Source`, `Kustomization`, `Release`, `Workload`, `ns/…`) to distinguish similarly named elements.



7.5. Platform Operations Graph

What this diagram shows: which platform actors operate which platform units and the control-plane targets they influence.

> **NOTE** Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.



7.6. Functional Groups and Units (View Scoped)

7.6.1. Payments (FG-PAYMENTS)

`payments` is deployed so `checkout` and authorization can roll independently with minimal blast radius. Deployment design prioritizes safe promotion, fast rollback, and dependency readiness checks before policy-impacting releases are exposed.

Checkout Handling (FU-CHECKOUT)

Checkout Handling Deployment Design

`checkout handling` is released independently to protect user-facing velocity without destabilizing policy logic. Deployment design favors fast rollback and compatibility checks with the current authorization interface.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-001`, `REQ-PAY-003`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-008`
Deployment dependencies	`Payment Authorization`
Inferred deployment evidence	runtime: `payments`, `payments/checkout-api`

Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Payment Authorization Deployment Design

`payment authorization` deployment is policy-sensitive and therefore rollout-cautious. Releases should stage gradually, verify dependency health, and preserve in-flight transaction continuity.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-001`, `REQ-PAY-004`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-009`
Deployment dependencies	`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`
Inferred deployment evidence	runtime: `payments/payment-engine`

7.6.2. Fraud Evaluation (FG-FRAUD)

`fraud evaluation` deployment is tuned for cautious rollout because scoring changes can alter approval rates. Deployment design favors staged promotions and rapid monitoring of scoring outcomes before wider exposure.

Risk Scoring (FU-RISK-SCORING)

Risk Scoring Deployment Design

`risk scoring` deployment uses staged rollout because scoring changes have immediate decision impact. Deployment design includes observation windows for score distribution and escalation-rate changes.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-002`, `REQ-PAY-005`
Deployment dependencies	`PostgreSQL Client Driver`

Deployment Aspect	View-Scoped Detail
Inferred deployment evidence	runtime: `risk`, `risk/risk-scorer`

Support Review (FU-SUPPORT-REVIEW)

Support Review Deployment Design

`support review` deployment isolates privileged workflows from public ingress components. Deployment design must preserve active review sessions across upgrades and ensure queue/review latency visibility.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-004`, `REQ-PAY-009`
Deployment dependencies	none
Inferred deployment evidence	authored unit with no direct derived runtime/code evidence yet

7.6.3. Platform (FG-PLATFORM)

`platform` deployment design governs how infrastructure intent is applied and kept in sync over time. It prioritizes deterministic sequencing, drift control, and recoverable change paths before application teams depend on new `platform` states.

Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Cluster Provisioning Deployment Design

`cluster provisioning` deployment is infrastructure-first and sequence-sensitive. Deployment design requires deterministic change plans, drift awareness, and rollback strategy before apply.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored `requirement` trigger
Deployment dependencies	none
Inferred deployment evidence	runtime: `payments`

GitOps Operations (FU-GITOPS-OPERATIONS)

GitOps Operations Deployment Design

`gitops operations` deployment defines the release control plane for application artifacts. Deployment design prioritizes policy gates, controlled promotion, and resilient rollback mechanics.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored `requirement` trigger
Deployment dependencies	`Cluster Provisioning`, `Helm` Runtime `Platform`
Inferred deployment evidence	runtime: `flux-system/payments-apps`, `flux-system/payments-sample-source`, `flux_system`

8. Security View

Show inferred exposure and dependency `risk` points aligned to unit boundaries, focused on attack paths and security evidence.

8.1. What This View Answers

- What are the main trust boundaries and attack paths?
- Which units are exposed and what security evidence exists?
- Where is the threat model incomplete?

8.2. Coverage Gaps

Coverage summary: 4/6 units have direct evidence coverage in this view.

- `Cluster Provisioning` has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.
- `GitOps Operations` has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.

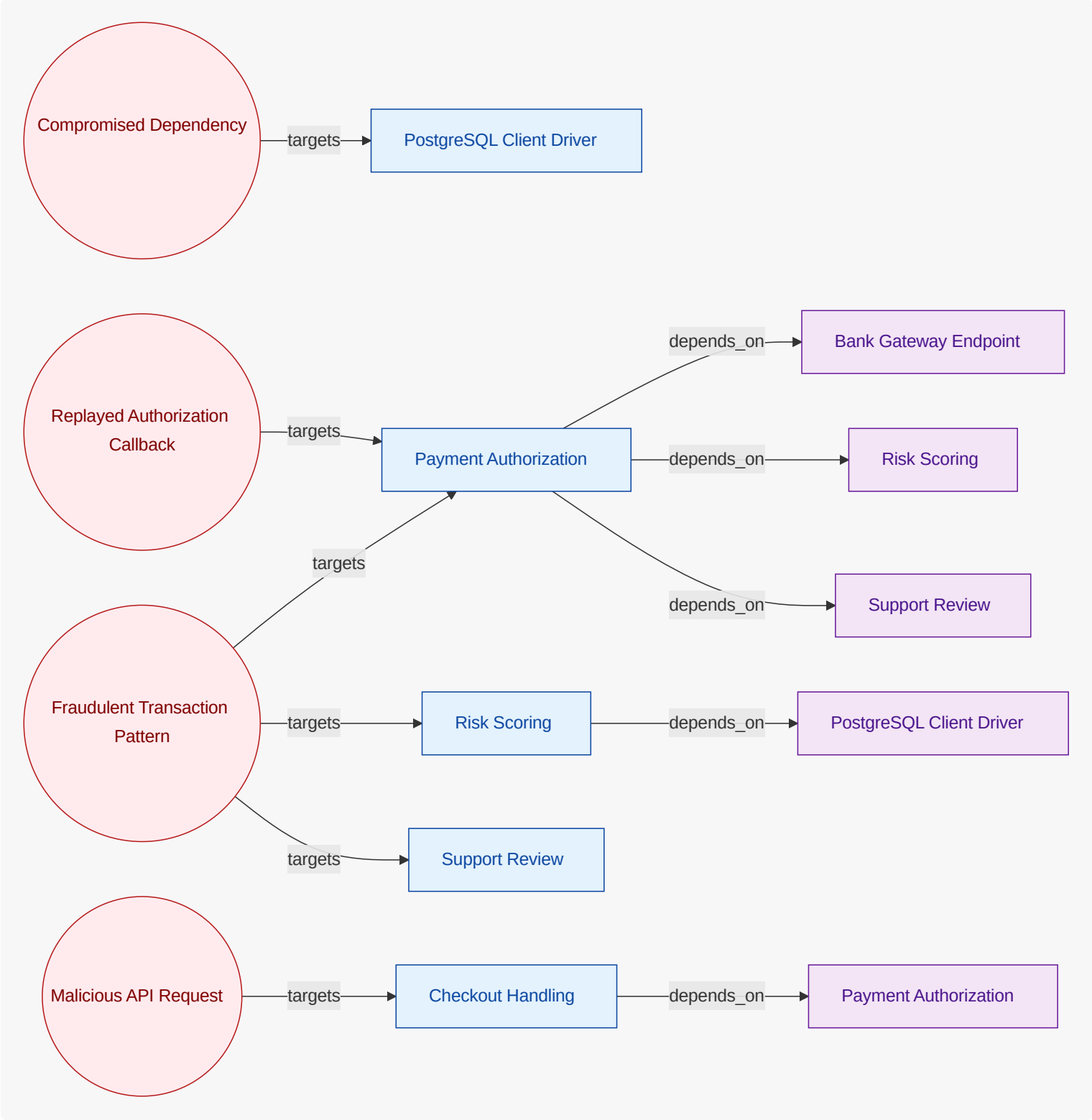
8.3. Recommended Next Evidence Additions

- Add explicit authored attack-vector mappings for units currently marked without threat linkage.
- Add security signal ownership metadata (logs/alerts/`events`) for unresolved observability evidence.
- Map additional security-relevant dependencies and exposures into `authored mappings` for traceable coverage.

8.4. Attack Path Diagram

What this diagram shows: authored attack vectors, their targeted units, and security-relevant dependency paths.

> **NOTE**
> Security relationships shown here are explicitly modeled or inferred from available evidence and may be incomplete. Absence of a path does not imply absence of risk. Threats that are currently mitigated (including fraud/abuse threats) should still be modeled as attack vectors for auditability and control-trace visibility.



8.5. Security Logging and Observability

Signal	Layer	Owner	Evidence
application security telemetry hooks	code	FU-RISK-SCORING	code support/audit_envelope.ts

Signal	Layer	Owner	Evidence
fraud and abuse detection code paths	code	FU-RISK-SCORING	code risk_scorer.ts
deployment and platform audit events	deployment	FU-CHECKOUT	helmrelease checkout-api
deployment and platform audit events	deployment	FU-CHECKOUT	namespace payments
deployment and platform audit events	deployment	FU-CLUSTER-PROVISIONING	cluster payments
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	gitrepository payments-sample-source
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	kustomization payments-apps
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	namespace flux_system
deployment and platform audit events	deployment	FU-PAYMENT-AUTHORIZATION	helmrelease payment-engine
deployment and platform audit events	deployment	FU-RISK-SCORING	helmrelease risk-scorer
deployment and platform audit events	deployment	FU-RISK-SCORING	namespace risk
abuse and fraud decision logs	runtime	FU-RISK-SCORING	helmrelease risk-scorer
abuse and fraud decision logs	runtime	FU-RISK-SCORING	namespace risk

8.6. Functional Groups and Units (View Scoped)

8.6.1. Payments (FG-PAYMENTS)

`payments` protects public trust boundaries where untrusted input and external callbacks enter the system. Security design focuses on strict validation, replay resistance, and safe failure behavior that preserves `customer` trust during degraded dependency conditions.

Checkout Handling (FU-CHECKOUT)

`checkout handling` security protects the internet-facing ingress. The design requires strict validation, abuse resistance, and traceable rejection behavior so malicious inputs are contained without harming legitimate `customer` flows.

Attack Surface

`Payment Authorization`

Main Threats

`Malicious API Request`

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: `payments`, `payments/checkout-api` | code modules: `checkout_api`

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

Payment Authorization (FU-PAYMENT-AUTHORIZATION)

`payment authorization` security protects decision integrity and callback integrity. The design defends against replay, policy probing, and ambiguous error channels that could be abused.

Attack Surface

`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`

Main Threats

`Fraudulent Transaction Pattern`, `Replayed Authorization Callback`

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: `payments/payment-engine` | code modules: domain, `payment_engine`

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.6.2. Fraud Evaluation (FG-FRAUD)

`fraud evaluation` security focuses on decision integrity and review integrity. The design assumes attempted manipulation of `risk` inputs, dependency trust, and `manual review` workflows, and therefore requires strong evidence lineage and tamper-evident operator actions.

Risk Scoring (FU-RISK-SCORING)

`risk scoring` security assumes adversarial input attempts and model manipulation pressure. Design controls enforce strict input handling and evidence capture for post-event investigation.

Attack Surface

`PostgreSQL Client Driver`

Main Threats

`Fraudulent Transaction Pattern`

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: `risk`, `risk/risk-scorer` | code modules: `risk_scorer`, `support`

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

Support Review (FU-SUPPORT-REVIEW)

`support review` security requires strong operator identity controls and tamper-evident audit trails. The design prevents silent overrides and guarantees every manual decision is attributable.

Attack Surface

none

Main Threats

`Fraudulent Transaction Pattern`

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

authored unit with no direct derived runtime/code evidence yet

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

8.6.3. Platform (FG-PLATFORM)

`platform` security design establishes baseline controls inherited by all higher-layer units. It focuses on trusted reconciliation, isolation defaults, and policy enforcement points where compromise would have broad systemic impact.

Cluster Provisioning (FU-CLUSTER-PROVISIONING)

`cluster provisioning` security sets inherited baseline hardening and isolation defaults. Design must detect and remediate security drift early because downstream controls depend on this layer.

Attack Surface

none

Main Threats

no explicit `attack vector` targeting this unit

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: `payments`

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

GitOps Operations (FU-GITOPS-OPERATIONS)

`gitops operations` security protects source trust and reconciliation authority. The design prevents unauthorized state changes and enforces policy before runtime apply occurs.

Attack Surface

`Cluster Provisioning`, `Helm` Runtime `Platform`

Main Threats

no explicit `attack vector` targeting this unit

Expected Controls

Define control expectations in authored security narratives and mapped dependencies.

Security Evidence Present

runtime: `flux-system/payments-apps`, `flux-system/payments-sample-source`, `flux_system`

Security Evidence Missing

Close any unresolved ownership or unmapped threat-linkage gaps in Traceability View.

9. Traceability View

Show requirement-to-unit-to-evidence traceability, including coverage confidence and explicit evidence gaps.

9.1. What This View Answers

- Which `requirements` map to which units?
- Which authored units have runtime/code evidence and which do not?
- Where are confidence or completeness gaps in trace coverage?

9.2. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view.

- `Cluster Provisioning` has no direct inferred code evidence yet.
- `GitOps Operations` has no direct inferred code evidence yet.
- `Support Review` has no direct inferred code evidence yet.
- `Support Review` has no direct inferred runtime evidence yet.

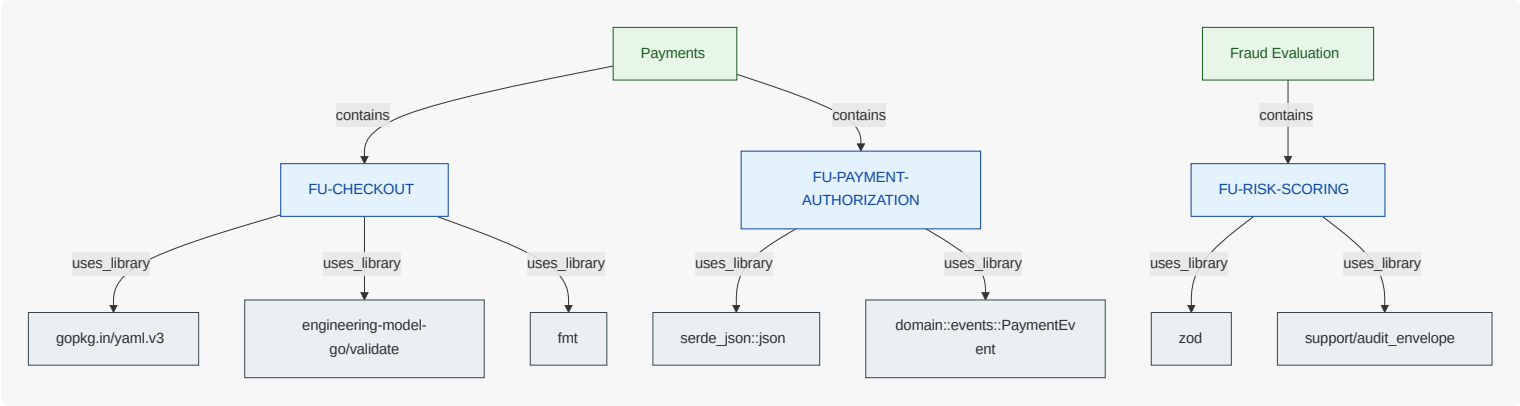
9.3. Recommended Next Evidence Additions

- Add explicit requirement-to-owner mappings where `requirement` scope is currently broad or ambiguous.
- Add ownership annotations for runtime and code artifacts where coverage is still unresolved.
- Prioritize closing gaps for units that are authored but lack both runtime and code evidence.

9.4. Traceability Evidence Graph

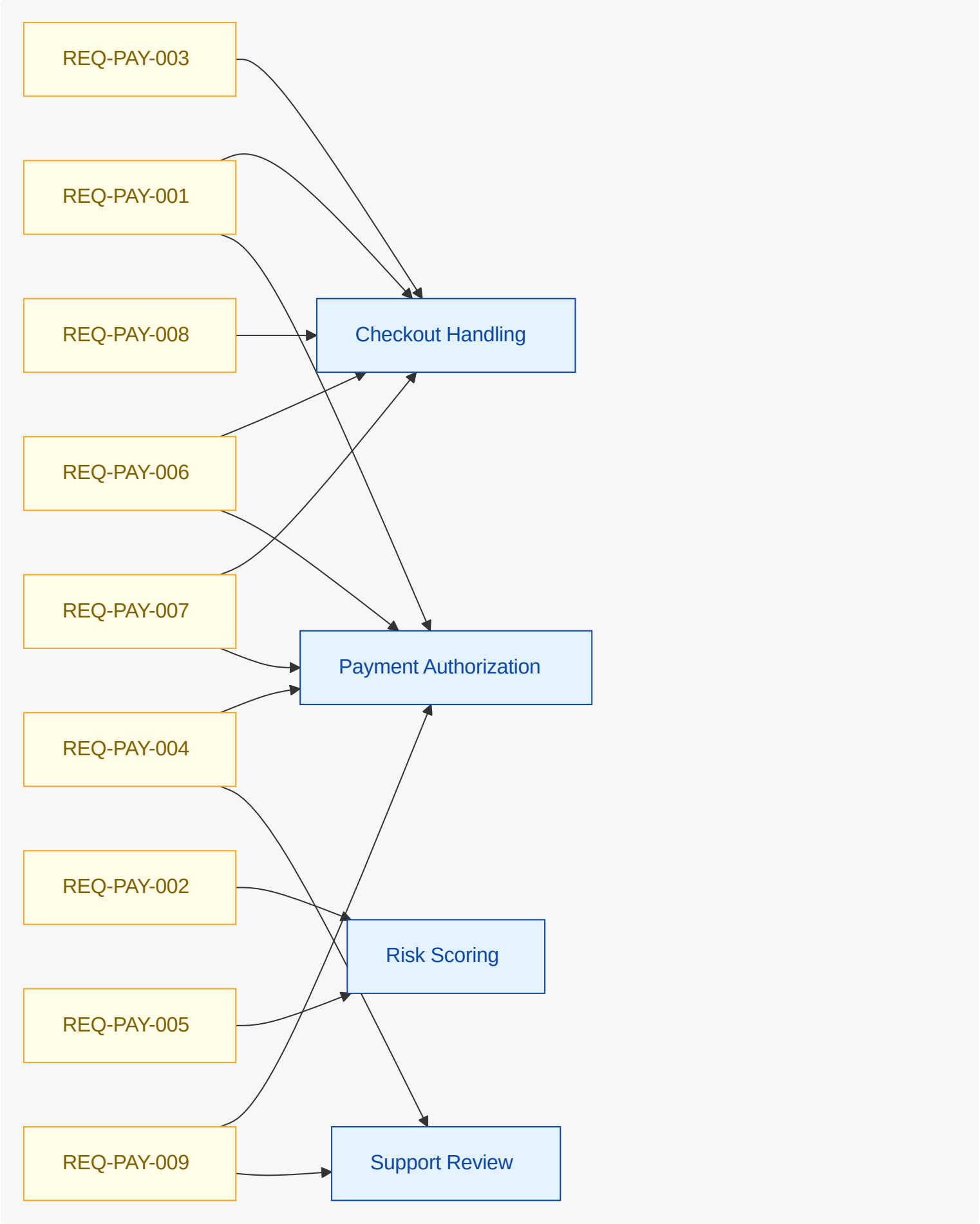
What this diagram shows: authored units and the inferred evidence links that support traceability confidence.

> **NOTE** This graph summarizes inferred ownership/evidence links and is intended for traceability confidence checks.



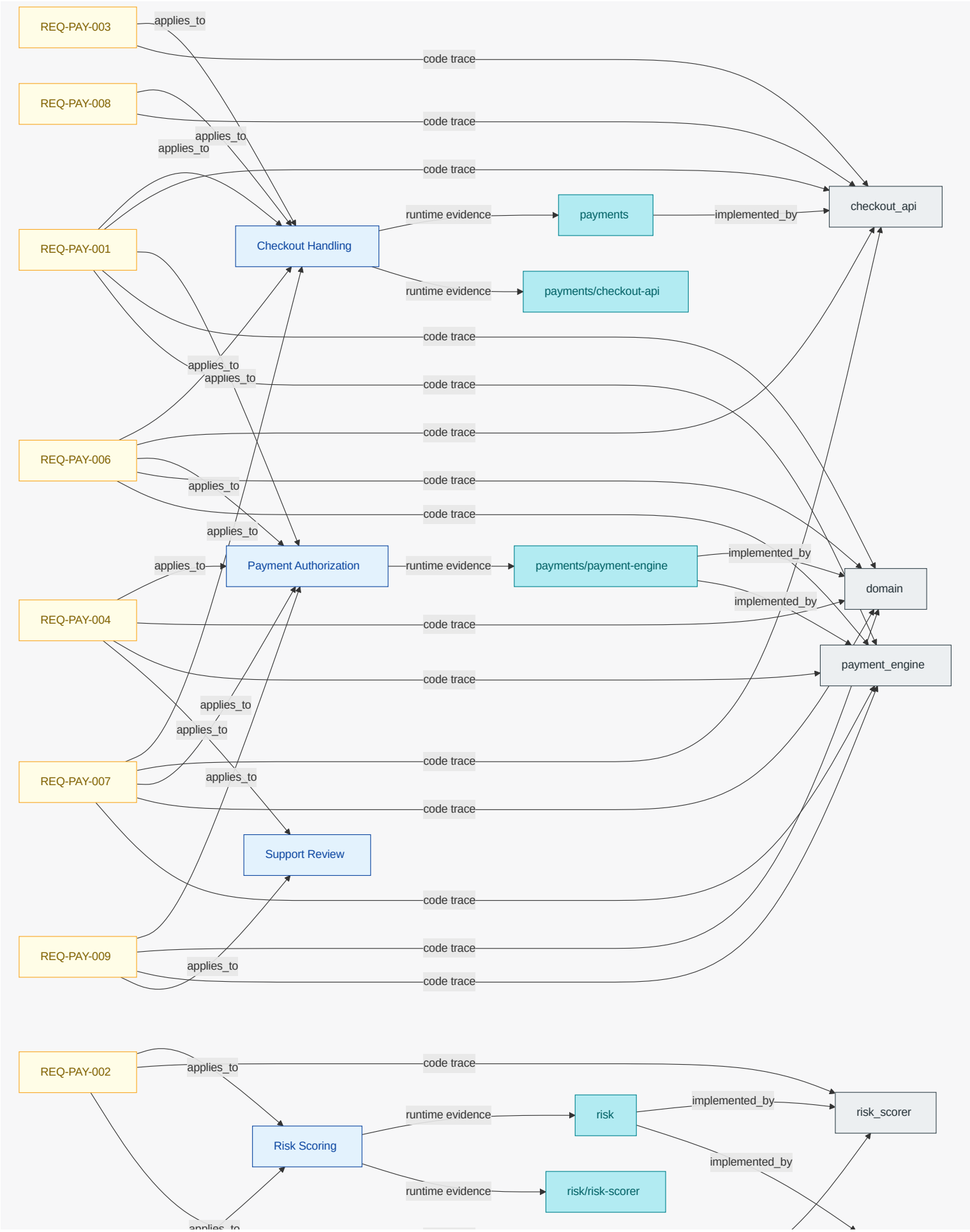
9.5. Requirement-to-Unit Mapping

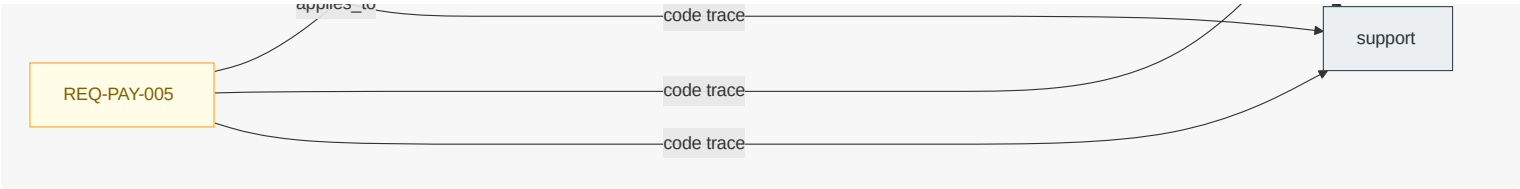
What this diagram shows: direct authored mapping from requirements to the functional units they apply to.


REQ-PAY-003
REQ-PAY-001
REQ-PAY-008
REQ-PAY-006
REQ-PAY-007
REQ-PAY-004
REQ-PAY-002
REQ-PAY-005
REQ-PAY-009
Checkout Handling
Payment Authorization
Risk Scoring
Support Review

9.6. Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: requirement-to-unit links extended with inferred runtime/code evidence attached to each unit.


REQ-PAY-003
REQ-PAY-008
REQ-PAY-001
REQ-PAY-006
REQ-PAY-004
REQ-PAY-007
REQ-PAY-009
REQ-PAY-002
applies_to
code trace
Checkout Handling
Payment Authorization
Support Review
Risk Scoring
runtime evidence
payments
payments/checkout-api
payments/payment-engine
risk
risk/risk-scorer
implemented_by
checkout_api
domain
payment_engine
risk_scorer



Show requirement-to-unit mappings inferred from appliesTo and authored architecture ownership boundaries.

10. Traceability Appendix

10.1. Requirement Details

10.1.1. REQ-PAY-001

While `checkout is active`, when `payment authorization is requested`, `the payments system` shall create a payment session. Notes: Core authorization path from `checkout` to bank gateway.

10.1.2. REQ-PAY-002

Where `fraud check is enabled`, when `payment authorization is requested`, `the fraud detection system` shall calculate a `risk score` before approval. Notes: `Fraud scoring` gate before approval.

10.1.3. REQ-PAY-003

If `payment is declined`, then `the payments system` shall present a `decline reason` to `customer`. Notes: `Decline` handling and `customer` feedback.

10.1.4. REQ-PAY-004

While `checkout is active`, when `payment authorization is requested` and `risk score is high`, `the payments system` shall place the payment in `payment review is pending`. Notes: Escalation flow for high-risk `payments`.

10.1.5. REQ-PAY-005

Where `fraud check is enabled`, when `payment authorization is requested`, `the payments system` shall persist an audit record with `payment id` plus `risk score`. Notes: Auditability for authorization decisions.

10.1.6. REQ-PAY-006

If `bank link is unavailable`, then `the payments system` shall present a temporary unavailable message to `customer`. Notes: Fallback behavior when external banking path is down.

10.1.7. REQ-PAY-007

While `checkout is active`, when `payment authorization is requested`, `the payments system` shall request authorization from `bank gateway endpoint`. Notes: External authorization call contract.

10.1.8. REQ-PAY-008

If `payment is declined`, then `the payments system` shall present a retry option to `customer`. Notes: `Customer` retry path `requirement` after `decline`.

10.1.9. REQ-PAY-009

While `checkout is active`, when `payment authorization is requested` and `risk score is high`, `the payments system` shall notify `support agent`. Notes: `Support` notification `requirement` for high-risk review.

10.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

10.2.1. Authorized Checkout Flow

Field	Value
ID	`VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597`
Kind	e2e
Status	not-run
Verifies Requirements	`REQ-PAY-001`
Test Code Elements	tests/e2e/authorized_checkout_flow.yaml
Derived Functional Owners	`FU-CHECKOUT`, `FU-PAYMENT-AUTHORIZATION`
Result Summary	not-run:1
Evidence	tests/e2e/authorized_checkout_flow.yaml
Notes	executes `authorized checkout flow` from session start through `payment authorization` success

10.2.2. Audit Record Persistence Test

Field	Value
ID	`VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2`
Kind	integration
Status	partial
Verifies Requirements	`REQ-PAY-005`
Test Code Elements	tests/integration/audit_record_persistence_test.ts
Derived Functional Owners	`FU-RISK-SCORING`
Result Summary	partial:1
Evidence	test-results/audit-persistence-report.md, tests/integration/audit_record_persistence_test.ts
Notes	checks audit record persistence includes `payment id` and computed `risk score`

10.2.3. Fraud Gate Test

Field	Value
ID	`VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD`
Kind	integration

Field	Value
Status	pass
Verifies Requirements	`REQ-PAY-002`, `REQ-PAY-004`
Test Code Elements	tests/integration/fraud_gate_test.rs
Derived Functional Owners	`FU-PAYMENT-AUTHORIZATION`, `FU-RISK-SCORING`, `FU-SUPPORT-REVIEW`
Result Summary	pass:2
Evidence	test-results/integration-fraud-gate.json, tests/integration/fraud_gate_test.rs
Notes	validates fraud gate escalation and block decisions before authorization completion

10.2.4. Contract Checkout Decline

Field	Value
ID	`VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26`
Kind	test
Status	pass
Verifies Requirements	`REQ-PAY-003`, `REQ-PAY-006`
Test Code Elements	none
Derived Functional Owners	`FU-CHECKOUT`, `FU-PAYMENT-AUTHORIZATION`
Result Summary	pass:2
Evidence	test-results/contract-checkout-decline.json
Notes	Inferred from test result artifact.

10.3. Verification Result Mapping

What this section shows: per-requirement result mapping inferred from test-result artifacts.

Check ID	Requirement	Result	Evidence / Notes
`VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597`	`REQ-PAY-001`	not-run	none No parsed test-result artifact for this `requirement` yet.

Check ID	Requirement	Result	Evidence / Notes
`VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD`	`REQ-PAY-002`	pass	test-results/integration-fraud-gate.json
`VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26`	`REQ-PAY-003`	pass	test-results/contract-checkout-decline.json
`VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD`	`REQ-PAY-004`	pass	test-results/integration-fraud-gate.json
`VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2`	`REQ-PAY-005`	partial	test-results/audit-persistence-report.md
`VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26`	`REQ-PAY-006`	pass	test-results/contract-checkout-decline.json

11. Reference Index

11.1. Authored References

REQ-PAY-001

While checkout is active, when payment authorization is requested, the payments system shall create a payment session.

Key	Value
Kind	Requirement
Name	REQ-PAY-001

REQ-PAY-002

Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval.

Key	Value
Kind	Requirement
Name	REQ-PAY-002

REQ-PAY-003

If payment is declined, then the payments system shall present a decline reason to customer.

Key	Value
Kind	Requirement
Name	REQ-PAY-003

REQ-PAY-004

While checkout is active, when payment authorization is requested and risk score is high, the payments system shall place the payment in payment review is pending.

Key	Value
Kind	Requirement
Name	REQ-PAY-004

REQ-PAY-005

Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id plus risk score.

Key	Value
Kind	Requirement
Name	REQ-PAY-005

REQ-PAY-006

If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer.

Key	Value
Kind	Requirement
Name	REQ-PAY-006

REQ-PAY-007

While checkout is active, when payment authorization is requested, the payments system shall request authorization from bank gateway endpoint.

Key	Value
Kind	Requirement
Name	REQ-PAY-007

REQ-PAY-008

If payment is declined, then the payments system shall present a retry option to customer.

Key	Value
Kind	Requirement
Name	REQ-PAY-008

REQ-PAY-009

While checkout is active, when payment authorization is requested and risk score is high, the payments system shall notify support agent.

Key	Value
Kind	Requirement
Name	REQ-PAY-009

11.2. Catalog References

ACT-CUSTOMER

End user submitting payment details during checkout.

Key	Value
Kind	Actor
Name	customer

ACT-PLATFORM-OPERATOR

Role responsible for GitOps and platform lifecycle operations.

Key	Value
Kind	Actor
Name	platform operator

ACT-SUPPORT

Operations role that investigates declines and high-risk review cases.

Key	Value
Kind	Actor
Name	support agent

AV-COMPROMISED-DEPENDENCY

Supply-chain compromise in runtime image or code dependency.

Key	Value
Kind	Attack Vector
Name	compromised dependency

AV-FRAUDULENT-TRANSACTION-PATTERN

Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals.

Key	Value
Kind	Attack Vector
Name	fraudulent transaction pattern

AV-MALICIOUS-API-REQUEST

Crafted payload attempting to bypass validation or authorization logic.

Key	Value
Kind	Attack Vector
Name	malicious api request

AV-REPLAYED-AUTH-CALLBACK

Replay of a valid callback to induce duplicate or unauthorized state transitions.

Key	Value
Kind	Attack Vector
Name	replayed authorization callback

COND-3DS-REQUIRED

Condition indicating strong customer authentication must be performed.

Key	Value
Kind	Condition
Name	3DS verification is required

COND-RISK-THRESHOLD-EXCEEDED

Condition indicating risk score crossed a policy threshold for escalation.

Key	Value
Kind	Condition
Name	risk threshold is exceeded

DATA-DECLINE-REASON

Structured reason code or message explaining why authorization failed.

Key	Value
Kind	Data Term
Name	decline reason

DATA-PAYMENT-ID

Stable identifier used to correlate a payment across services and records.

Key	Value
Kind	Data Term
Name	payment id

DATA-RISK-SCORE

Numeric fraud score produced by risk evaluation logic.

Key	Value
Kind	Data Term
Name	risk score

EM-ACTOR

A person or operational role that interacts with functional units.

Key	Value
Kind	Engineering Model Term
Name	actor

EM-ATTACK-VECTOR

A technical misuse or attack path that targets functional, referenced, or runtime elements.

Key	Value
Kind	Engineering Model Term
Name	attack vector

EM-AUTHORED-MAPPING

An explicit relationship declared in architecture inputs between authored or referenced elements.

Key	Value
Kind	Engineering Model Term
Name	authored mapping

EM-CODE-ELEMENT

An inferred code structure or ownership element discovered from source trees and build metadata.

Key	Value
Kind	Engineering Model Term
Name	code element

EM-FUNCTIONAL-GROUP

A major authored capability area that groups related functional units.

Key	Value
Kind	Engineering Model Term
Name	functional group

EM-FUNCTIONAL-UNIT

An authored working unit inside a functional group that owns specific behavior.

Key	Value
Kind	Engineering Model Term
Name	functional unit

EM-INFERRED-MAPPING

A discovered relationship that links inferred runtime/code elements upward to authored design.

Key	Value
Kind	Engineering Model Term
Name	inferred mapping

EM-REFERENCED-ELEMENT

An architecture-relevant external, platform, or third-party dependency represented by role.

Key	Value
Kind	Engineering Model Term
Name	referenced element

EM-REQUIREMENT

A structured requirement statement that defines expected system behavior and maps to functional ownership.

Key	Value
Kind	Engineering Model Term
Name	requirement

EM-RUNTIME-ELEMENT

An inferred runtime realization element discovered from infrastructure and deployment sources.

Key	Value
Kind	Engineering Model Term
Name	runtime element

EM-UPWARD-LINKING

Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

Key	Value
Kind	Engineering Model Term
Name	upward linking

EM-VERIFICATION-CHECK

An inferred or authored verification artifact that validates requirement behavior with test evidence.

Key	Value
Kind	Engineering Model Term
Name	verification check

EVT-BANK-LINK-UNAVAILABLE

Event raised when payment engine cannot reach the external bank gateway.

Key	Value
Kind	Event
Name	bank link is unavailable

EVT-PAYMENT-AUTH-REQUESTED

Event raised when checkout asks payment engine to authorize a transaction.

Key	Value
Kind	Event
Name	payment authorization is requested

EVT-PAYMENT-DECLINED

Event indicating authorization was rejected and customer decline handling must execute.

Key	Value
Kind	Event
Name	payment is declined

EVT-RISK-HIGH

Event emitted when computed fraud risk exceeds the configured high-risk threshold.

Key	Value
Kind	Event
Name	risk score is high

EVT-SUPPORT-NOTIFIED

Event indicating manual review notification was sent to support operations.

Key	Value
Kind	Event
Name	support agent is notified

FEAT-FRAUD-CHECK

Feature flag controlling whether risk scoring is required before authorization.

Key	Value
Kind	Feature
Name	fraud check is enabled

FG-FRAUD

Functional group for risk scoring and fraud review capabilities.

Key	Value
Kind	Functional Group
Name	fraud evaluation

FG-PAYMENTS

Functional group for checkout and authorization capabilities.

Key	Value
Kind	Functional Group
Name	payments

FG-PLATFORM

Functional group for provisioning and platform operations capabilities.

Key	Value
Kind	Functional Group
Name	platform

FU-CHECKOUT

Handles checkout session start and customer-facing payment messaging.

Key	Value
Kind	Functional Unit
Name	checkout handling

FU-CLUSTER-PROVISIONING

Provides cluster-level platform setup and environment bootstrap.

Key	Value
Kind	Functional Unit
Name	cluster provisioning

FU-GITOPS-OPERATIONS

Operates declarative delivery and release management flows.

Key	Value
Kind	Functional Unit
Name	gitops operations

FU-PAYMENT-AUTHORIZATION

Coordinates authorization flow and bank gateway interaction.

Key	Value
Kind	Functional Unit
Name	payment authorization

FU-RISK-SCORING

Evaluates transaction risk and returns score classification.

Key	Value
Kind	Functional Unit
Name	risk scoring

FU-SUPPORT-REVIEW

Supports manual review and escalation handling for risky transactions.

Key	Value
Kind	Functional Unit
Name	support review

MODE-NORMAL

Standard operating mode with full external dependency usage enabled.

Key	Value
Kind	Mode
Name	normal mode is active

REF-BANK-GATEWAY-ENDPOINT

External banking endpoint used for card authorization and decline decisions.

Key	Value
Kind	Referenced Element
Name	bank gateway endpoint

REF-HELM-PLATFORM

Runtime packaging and deployment substrate for workloads.

Key	Value
Kind	Referenced Element
Name	helm platform

REF-POSTGRES-DRIVER

Third-party library used for application data persistence connectivity.

Key	Value
Kind	Referenced Element
Name	postgresql client driver

STATE-BANK-LINK-UP

Connectivity state showing bank gateway communication is currently healthy.

Key	Value
Kind	State
Name	bank link is available

STATE-CHECKOUT-ACTIVE

Processing state where the user is actively submitting or confirming payment.

Key	Value
Kind	State
Name	checkout is active

STATE-REVIEW-PENDING

Transaction state requiring manual support review before final disposition.

Key	Value
Kind	State
Name	payment review is pending

SYS-FRAUD-DETECTION-SYSTEM

Fraud scoring and review subsystem participating in payment decisions.

Key	Value
Kind	System
Name	fraud detection system

SYS-PAYMENTS-SYSTEM

Payment processing system-of-interest covered by this architecture.

Key	Value
Kind	System
Name	payments system

11.3. Inferred Runtime References

payments

provisions the shared EKS control plane that hosts payments and risk workloads

Key	Value
Kind	Runtime cluster
Owner	FU-CLUSTER-PROVISIONING
Source	examples/payments-engineering-sample/infra/terraform/main.tf

flux-system/payments-sample-source

tracks the desired repository state that drives Flux reconciliation for payments workloads

Key	Value
Kind	Runtime gitrepository
Owner	FU-GITOPS-OPERATIONS
Source	examples/payments-engineering-sample/infra/flux/sources/payments-source.yaml

payments/checkout-api

exposes checkout ingress and forwards normalized payment requests to authorization flow

Key	Value
Kind	Runtime helmrelease
Owner	FU-CHECKOUT
Source	examples/payments-engineering-sample/infra/flux/releases/checkout-api.yaml

payments/payment-engine

executes payment authorization decisions and coordinates fraud checks plus bank callbacks

Key	Value
Kind	Runtime helmrelease
Owner	FU-PAYMENT-AUTHORIZATION
Source	examples/payments-engineering-sample/infra/flux/releases/payment-engine.yaml

risk/risk-scorer

computes fraud risk score signals and persists evidence used by authorization decisions

Key	Value
Kind	Runtime helmrelease
Owner	FU-RISK-SCORING
Source	examples/payments-engineering-sample/infra/flux/releases/risk-scorer.yaml

flux-system/payments-apps

applies and reconciles HelmRelease definitions for checkout, payment, and risk services

Key	Value
Kind	Runtime kustomization
Owner	FU-GITOPS-OPERATIONS
Source	examples/payments-engineering-sample/infra/flux/kustomizations/payments-apps.yaml

flux_system

control-plane namespace where Flux GitOps operators reconcile deployment state

Key	Value
Kind	Runtime namespace
Owner	FU-GITOPS-OPERATIONS
Source	examples/payments-engineering-sample/infra/terraform/main.tf

payments

runtime namespace boundary for checkout and payment authorization workloads

Key	Value
Kind	Runtime namespace
Owner	FU-CHECKOUT
Source	examples/payments-engineering-sample/infra/terraform/main.tf

risk

runtime namespace boundary for fraud scoring and review support workloads

Key	Value
Kind	Runtime namespace
Owner	FU-RISK-SCORING
Source	examples/payments-engineering-sample/infra/terraform/main.tf

11.4. Inferred Code References

gopkg.in/yaml.v3

Inferred library_external dependency owned by FU-CHECKOUT from checkout_api.go.

Key	Value
Kind	Code library_external
Owner	FU-CHECKOUT
Source	checkout_api.go

serde_json::json

Inferred library_external dependency owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs.

Key	Value
Kind	Code library_external
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs

zod

Inferred library_external dependency owned by FU-RISK-SCORING from risk_scorer.ts.

Key	Value
Kind	Code library_external
Owner	FU-RISK-SCORING
Source	risk_scorer.ts

./support/audit_envelope

Inferred library_first_party dependency owned by FU-RISK-SCORING from risk_scorer.ts.

Key	Value
Kind	Code library_first_party
Owner	FU-RISK-SCORING
Source	risk_scorer.ts

crate::domain::events::PaymentEvent

Inferred library_first_party dependency owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs.

Key	Value
Kind	Code library_first_party
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs

github.com/labeth/engineering-model-go/validate

Inferred library_first_party dependency owned by FU-CHECKOUT from checkout_api.go.

Key	Value
Kind	Code library_first_party

Key	Value
Owner	FU-CHECKOUT
Source	checkout_api.go

fmt

Inferred library_stdlib dependency owned by FU-CHECKOUT from checkout_api.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-CHECKOUT
Source	checkout_api.go

checkout_api.go

accepts checkout requests and forwards payment authorization flow input

Key	Value
Kind	Code source_file
Owner	FU-CHECKOUT
Source	checkout_api.go

domain/events.rs

defines payment domain events used across authorization and risk paths

Key	Value
Kind	Code source_file
Owner	FU-PAYMENT-AUTHORIZATION
Source	domain/events.rs

payment_engine.rs

executes authorization decisions and persists payment-side audit records

Key	Value
Kind	Code source_file
Owner	FU-PAYMENT-AUTHORIZATION

Key	Value
Source	payment_engine.rs

risk_scorer.ts

computes fraud risk scores and decides review or decline outcomes

Key	Value
Kind	Code source_file
Owner	FU-RISK-SCORING
Source	risk_scorer.ts

support/audit_envelope.ts

formats support-review audit envelopes for traceability workflows

Key	Value
Kind	Code source_file
Owner	FU-RISK-SCORING
Source	support/audit_envelope.ts

CODE-AUTHORIZEPAYMENT

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:11.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:11

CODE-BUILDREVIEWREASON

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:50.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:50

CODE-CALCULATERISKSCORE

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:8.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:8

CODE-CLASSIFYRISK

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:21.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:21

CODE-CREATEAUDITRECORD

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:37.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:37

CODE-ENRICHAUDITMETADATA

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:45.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:45

CODE-HANDLEBANKLINKUNAVAILABLE

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:34.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:34

CODE-HANDLEBANKUNAVAILABLE

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:32.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:32

CODE-ISHIGHRISK

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:29.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:29

CODE-NOTIFYSUPPORT

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:52.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:52

CODE-PERSISTAUDITRECORD

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:60.

Key	Value
Kind	Code symbol

Key	Value
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:60

CODE-PLACEINREVIEW

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:44.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:44

CODE-REQUESTBANKAUTHORIZATION

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:26.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:26

CODE-SHOWDECLINEREASON

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:37.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:37

CODE-SHOWRETRYOPTION

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:42.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT

Key	Value
Source	checkout_api.go:42

CODE-SHOWTEMPORARYUNAVAILABLE

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:47.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:47

CODE-STARTSESSION

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:16.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:16

CODE-SUBMITAUTHORIZATIONREQUEST

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:27.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:27

11.5. Verification References

VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597

executes authorized checkout flow from session start through payment authorization success

Key	Value
Kind	e2e
Status	not-run

Key	Value
Source	tests/e2e/authorized_checkout_flow.yaml
Name	Authorized Checkout Flow

VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2

checks audit record persistence includes payment id and computed risk score

Key	Value
Kind	integration
Status	partial
Source	test-results/audit-persistence-report.md (+1)
Name	Audit Record Persistence Test

VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD

validates fraud gate escalation and block decisions before authorization completion

Key	Value
Kind	integration
Status	pass
Source	test-results/integration-fraud-gate.json (+1)
Name	Fraud Gate Test

VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26

Inferred from test result artifact.

Key	Value
Kind	test
Status	pass
Source	test-results/contract-checkout-decline.json
Name	Contract Checkout Decline